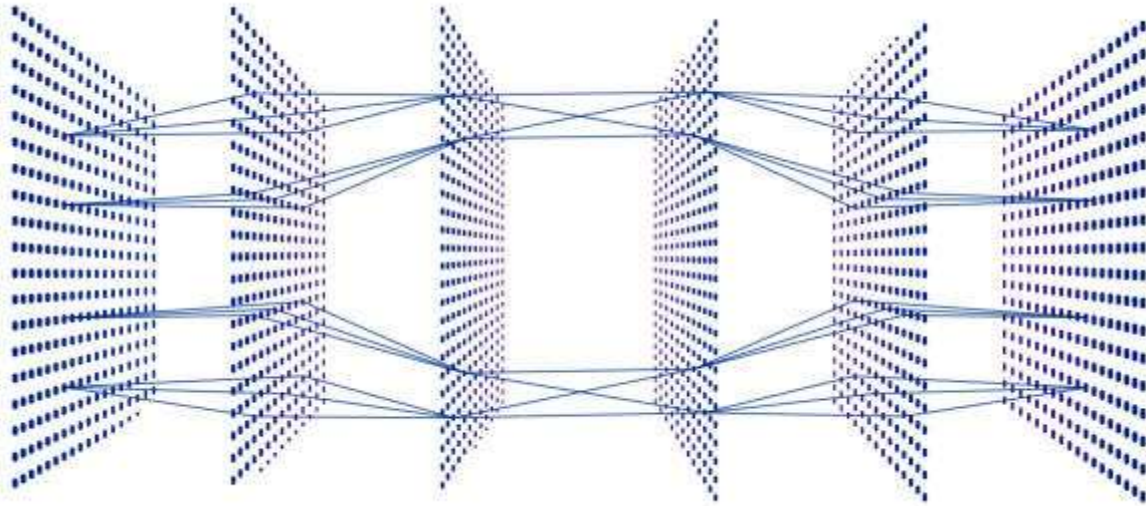


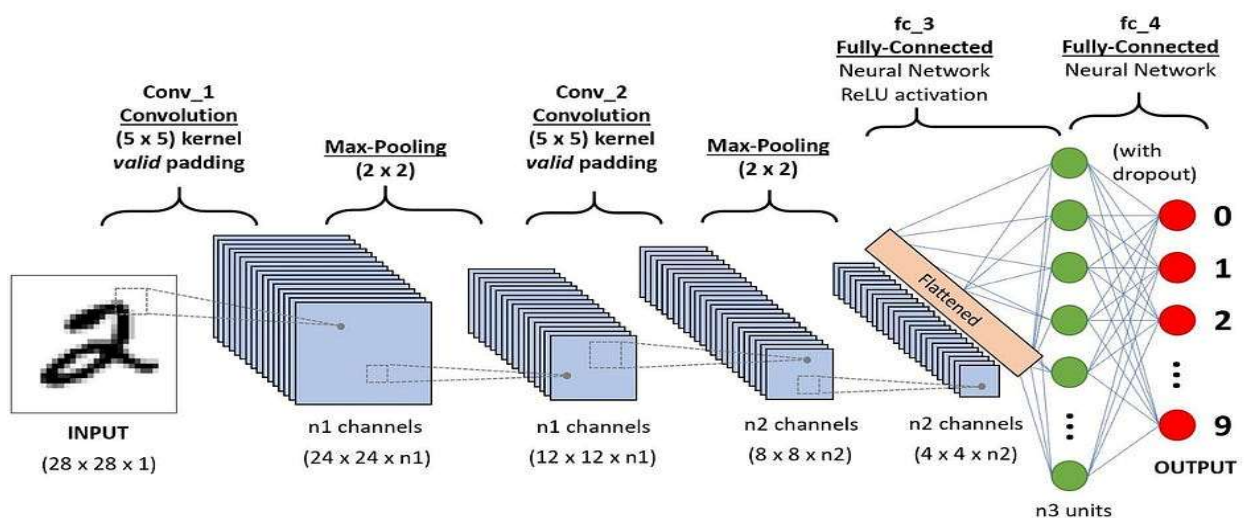
Convolutional Neural Networks (CNN) — Architecture Explained



Introduction

A convolutional neural network (CNN), is a network architecture for deep learning which learns directly from data. CNNs are particularly useful for finding patterns in images to recognize objects. They can also be quite effective for classifying non-image data such as audio, time series, and signal data.

[Press enter or click to view image in full size](#)



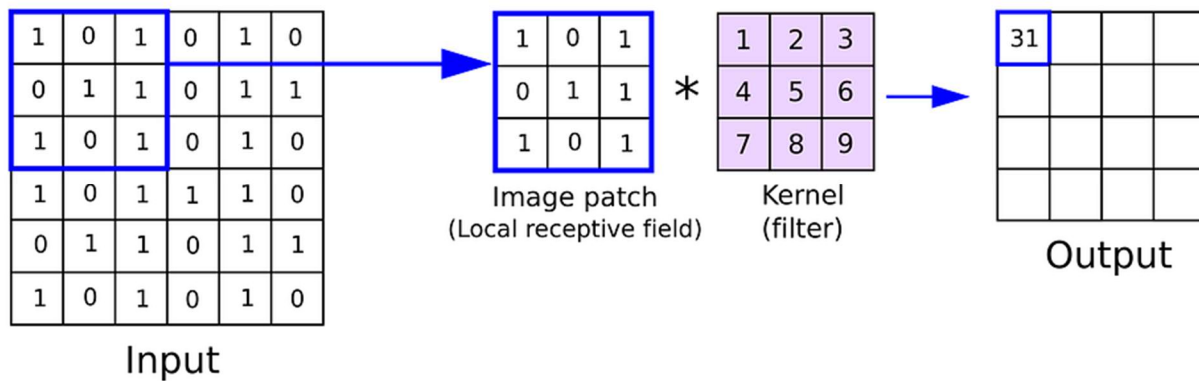
Kernel or Filter or Feature Detectors

In a convolutional neural network, the kernel is nothing but **a filter that is used to extract the features from the images.**

$$\text{Formula} = [i-k]+1$$

i -> Size of input , K-> Size of kernel

Press enter or click to view image in full size



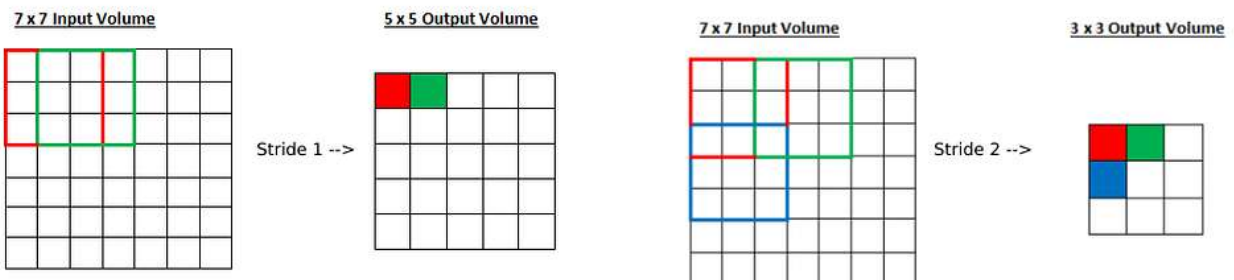
Stride

Stride is a parameter of the neural network's filter that modifies the amount of movement over the image or video. we had stride 1 so it will take one by one. If we give stride 2 then it will take value by skipping the next 2 pixels.

$$\text{Formula} = [i-k/s]+1$$

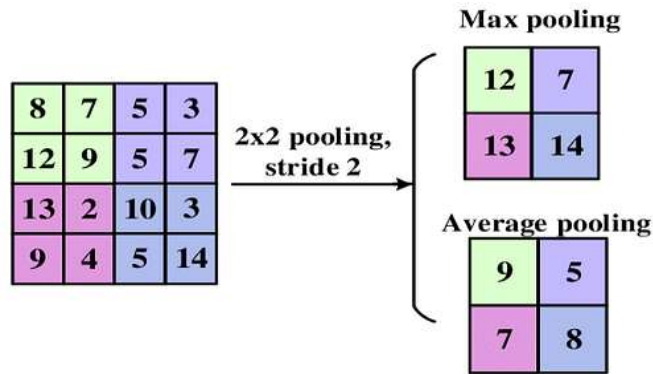
i -> Size of input , K-> Size of kernel, S-> Stride

Press enter or click to view image in full size



Padding

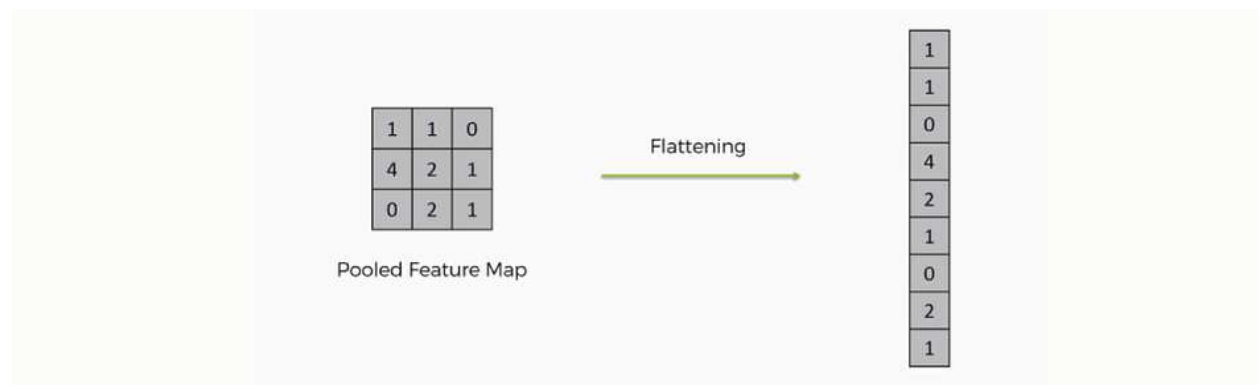
Press enter or click to view image in full size



Flatten

Flattening is used to convert all the resultant 2-Dimensional arrays from pooled feature maps into a single long continuous linear vector. The flattened matrix is fed as input to the fully connected layer to classify the image.

Press enter or click to view image in full size



Layers used to build CNN

Convolutional neural networks are distinguished from other neural networks by their superior performance with image, speech, or audio signal inputs. They have three main types of layers, which are:

- Convolutional layer
- Pooling layer
- Fully-connected (FC) layer

Convolutional layer

This layer is the first layer that is used to extract the various features from the input images. In this layer, We use a filter or Kernel method to extract features from the input image.

$$\frac{W - F + 2P}{S} + 1$$

Pooling layer

The primary aim of this layer is to decrease the size of the convolved feature map to reduce computational costs. This is performed by decreasing the connections between layers and independently operating on each feature map. Depending upon the method used, there are several types of Pooling operations. We have Max pooling and average pooling.

$$\frac{W - F}{S} + 1$$

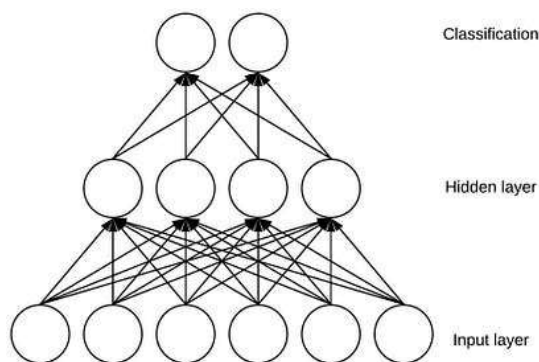
Fully-connected layer

The Fully Connected (FC) layer consists of the weights and biases along with the neurons and is used to connect the neurons between two different layers. These layers are usually placed before the output layer and form the last few layers of a CNN Architecture.

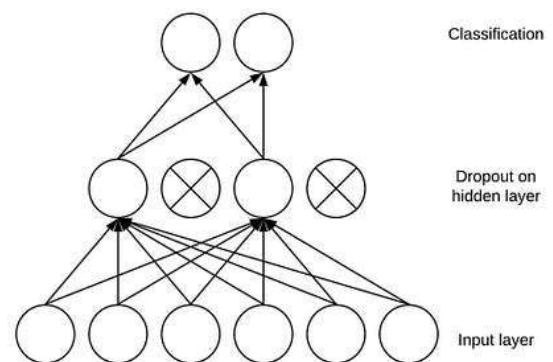
Dropout

Another typical characteristic of CNNs is a Dropout layer. The Dropout layer is a mask that nullifies the contribution of some neurons towards the next layer and leaves unmodified all others.

Press enter or click to view image in full size



Without Dropout



With Dropout

Activation Function

An Activation Function decides whether a neuron should be activated or not. This means that it will decide whether the neuron's input to the network is important or not in the process of prediction. There are several commonly used activation functions such as the ReLU, Softmax, tanH, and the Sigmoid functions. Each of these functions has a specific usage.

Sigmoid — For a binary classification in the CNN model

tanH - The tanh function is very similar to the sigmoid function. The only difference is that it is symmetric around the origin. The range of values, in this case, is from -1 to 1.

Softmax- It is used in multinomial logistic regression and is often used as the last activation function of a neural network to normalize the output of a network to a probability distribution over predicted output classes.

ReLU- the main advantage of using the ReLU function over other activation functions is that it does not activate all the neurons at the same time.

Zero-Padding in Convolutional Neural Networks

Introduction

Hello everyone! In this post, we are going to learn about what is Zero-Padding and why we are using Convolutional Neural Networks. Before we dive into padding let's discuss the kernel. The kernel is the neural networks filter that moves across the image, scanning each pixel and converting the data into a smaller, or sometimes larger, format. To assist the kernel with processing the image, padding is added to the frame of the image to allow for more space for the kernel to cover the image. Adding padding to an image processed by a CNN allows for a more accurate analysis of images.

Convolutions Reduce Channel Dimensions

The convolutional layer has several filters and those are applied to an input to create a feature map that summarizes the presence of detected features in the input. When a filter convolves a given input channel, it gives us an output channel. This output channel is a matrix of pixels with the values that were computed during the convolutions that occurred on the input channel. When this happens, the dimensions of our image are reduced. If zero-padding = 1, there will be one pixel thick around the original image with pixel value = 0.

Press enter or click to view image in full size

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

Zero-padding vs. scaling up using interpolation

There are two approaches to resize smaller images up to the fixed size: zero-padding and scaling them up (zooming in) using interpolation. Zero-padding has two advantages in comparison with scaling. The first advantage is that while scaling carries the risk of deforming the patterns in the image, padding does not. The second advantage of zero-padding is that it speeds up the calculations, in comparison with scaling, resulting in better computational efficiency. The reason is that neighboring zero input units (pixels) will not activate their corresponding convolutional unit in the next layer. Therefore, the synaptic weights on outgoing links from input units do not need to be updated if they contain a zero value. This is like a dropout that only concerns border pixels in the input layer. This advantage will be lost if smaller images are enlarged by increasing their resolution (scaling) rather than zero-padding.

Press enter or click to view image in full size



(a)



(b)



(c)

A small image (a) resized to 180×180 pixels using zero-padding (b) and interpolation (c)

How to calculate optimal zero-padding for convolutional neural networks?

The possible values for the padding size, P , depends on the input size, the filter size F , and the stride S . We assume width and height are the same. What you need to ensure is that the output size, $(W-F+2P)/S+1$, is an integer. When $S=1$ then you get your first equation $P=(F-1)/2$ as a necessary condition. But, in general, you need to consider the three parameters, namely W , F , and S to determine valid values of P .

Conclusion

Convolutional neural networks do not learn a single filter; they, in fact, learn multiple features in parallel for a given input. It is common for a convolutional layer to learn from 32 to 512 filters in parallel for a given input. We now understood what zero paddings is, what it achieves when we add it to our CNN.

Pooling and their types in CNN

Pooling is a fundamental operation in Convolutional Neural Networks (CNNs) that plays a crucial role in downsampling feature maps while retaining important information.

In this blog post, we'll explore the concept of pooling in CNNs, its significance, and the different types of pooling techniques used in practice.

Pooling, also known as subsampling or downsampling, is a technique used in CNNs to reduce the spatial dimensions of feature maps while retaining essential information. It helps in controlling the model's complexity, reducing overfitting, and improving computational efficiency by reducing the number of parameters and computation required in subsequent layers.

The pooling operation involves sliding a two-dimensional filter over each channel of feature map and summarising the features lying within the region covered by the filter.

For a feature map having dimensions **nh x nw x nc**, the dimensions of output obtained after a pooling layer is

$$(nh - f + 1) / s \times (nw - f + 1) / s \times nc$$

where,

-> nh - height of feature map

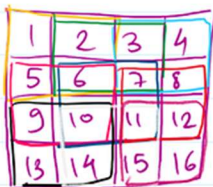
-> nw - width of feature map

-> nc - number of channels in the feature map

-> f - size of filter

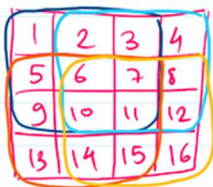
-> s - stride length

Press enter or click to view image in full size



1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

filter of size 2 by 2 with stride of 1 by 1



1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

filter of size 3 by 3 with stride of 1 by 1

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

filter of size 3 by 3 with stride of 2 by 2

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

filter of size 2 by 2 with stride of 2 by 2

Types of Pooling Layers:

Max Pooling

Max pooling is a pooling operation that selects the maximum element from the region of the feature map covered by the filter.

Thus, the output after max-pooling layer would be a feature map containing the most prominent features of the previous feature map.

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Max Pool

 Filter - (2 x 2)
 Stride - (2, 2)

9	7
8	6

Average Pooling

Average pooling computes the average of the elements present in the region of feature map covered by the filter.

Thus, while max pooling gives the most prominent feature in a particular patch of the feature map, average pooling gives the average of features present in a patch.



Global Pooling

Global pooling reduces each channel in the feature map to a single value. Thus, an $nh \times nw \times nc$ feature map is reduced to $1 \times 1 \times nc$ feature map. This is equivalent to using a filter of dimensions $nh \times nw$ i.e. the dimensions of the feature map.

Further, it can be either global max pooling or global average pooling.

